Day 3 Morning — Web Technologies

CSS Positioning • Login Form Styling • Flexbox Layout • JavaScript Introduction

Source: Oct 23 lecture transcript with your reference notes. Lecture order is preserved. Explanations marked “extra clarification” add useful precision beyond what was demonstrated in the transcript.

1. Build the HTML structure before styling

The instructor begins with a login-screen task and emphasizes the workflow: first create the HTML skeleton, then apply CSS. The login area is one container; inside it are a heading, a form with username and password fields, a login button, and a sign-up link.

```
<div id="login-container">
  <h2 id="login-heading">Login</h2>

  <form>
    <div><input class="form-input" type="text"
                placeholder="Enter username"></div>
    <div><input class="form-input" type="password"
                placeholder="Enter password"></div>
    <div><button id="login-button" type="submit">Login</button></div>
  </form>

  <p>Need account? <a id="signup-link" href="#">Sign up</a></p>
</div>
```

Important: For a real sign-up page, replace href="#" with the actual page URL, such as signup.html. A hash is a temporary placeholder, not a real destination.

2. Center the login container with margin auto

To center a fixed-width block horizontally, use margin-left: auto and margin-right: auto, commonly written as margin: 80px auto 0. The browser calculates equal left and right margins. This is different from text-align: center, which centers inline content inside a container.

```
#login-container {
  width: 400px;
  min-height: 400px;
  margin: 80px auto 0;
  background-color: white;
  text-align: center;
}
body {
  background-color: #3f51b5;
  font-family: "Segoe UI", Arial, sans-serif;
}
```

In the shorthand margin: 80px auto 0, the values mean top = 80px, left/right = auto, bottom = 0. This works for horizontal centering when the element has a constrained width. It does not by itself vertically center the box in the viewport.

3. Style the login fields and button

```
.form-input {
  width: 65%;
  margin-bottom: 25px;
  padding: 15px 10px;
  border: 0;
  border-bottom: 1px solid #e53935;
  outline: none;
}

#login-button {
  width: 30%;
  padding: 12px 10px;
  margin-top: 10px;
  border: 0;
  border-radius: 15px;
  background-color: #e53935;
  color: white;
  transition: background 1s;
}

#login-button:hover {
  background: white;
  color: #e53935;
  border: 1px solid #e53935;
}

#signup-link {
  text-decoration: none;
}
```

The shared class form-input styles both input fields without repeating CSS. Percentage widths are calculated relative to the containing block in this example, not automatically relative to the whole browser window.

border vs. outline: border belongs to the element's box. A browser may show an outline around a focused input; outline is separate from border. The instructor demonstrates outline: none, but extra clarification: removing the focus outline can make keyboard navigation harder. Prefer a visible custom focus style in real interfaces.

4. CSS hover and transition

:hover is a CSS pseudo-class that applies while a pointing device is over an element. It is not a JavaScript event handler. transition controls how smoothly supported CSS property changes animate over time.

```
#login-button {
  background-color: #e53935;
  color: white;
  transition: background-color 0.3s ease, color 0.3s ease;
}

#login-button:hover {
  background-color: white;
  color: #e53935;
}
```

The lecture uses the background shorthand with a duration. For learning, prefer listing the specific property, such as background-color, when that is the property you intend to animate. Hover styles should not be the only way to communicate important information.

5. CSS position: static, relative, absolute and fixed

The transcript introduces these position values and the offset properties top, left, right and bottom, asking students to explore them. The explanations below are a concise reference to help with that practice.

Position values

static — default positioning. The element follows normal document flow. top/left/right/bottom do not reposition a static element.

relative — the element stays in normal flow, but can be visually shifted from its original position using offsets. Its original space remains reserved.

absolute — the element is removed from normal flow and positioned relative to its nearest ancestor that establishes a positioning context (often an ancestor with position: relative). If none exists, positioning is relative to the initial containing block.

fixed — usually positioned relative to the browser viewport and stays in place while scrolling. Certain ancestor properties can change its containing block.

```
.parent {
  position: relative;
}

.badge {
  position: absolute;
  top: 10px;
  right: 10px;
}

.notice {
  position: fixed;
  bottom: 16px;
  right: 16px;
}

.shifted {
  position: relative;
  top: 8px;
  left: 12px;
}
```

Remember: top/left/right/bottom are offsets, not position modes by themselves. Their effect depends on the element's positioning mode. Use position: relative on a parent when you want an absolutely positioned child to be placed inside that parent.

6. Display and the parent-child relationship

The lecture first discusses display, including display: none to remove an element from layout, and then introduces display: flex. Flexbox is applied to a parent/container; its direct children become flex items. Applying it to a child does not automatically arrange that child alongside its siblings.

```
<div id="container">
  <div id="box1">div 1</div>
  <div id="box2">div 2</div>
  <div id="box3">div 3</div>
</div>
```

```
#container {
```

```
  display: flex;
  gap: 20px;
}
#box1 { background-color: red; }
#box2 { background-color: blue; }
#box3 { background-color: green; }
```

display: none hides an element and removes its layout space. This differs from visibility: hidden, which hides the element but generally preserves its space.

7. Flexbox: main axis, cross axis and alignment

With the default flex-direction: row, the main axis runs horizontally from left to right, and the cross axis runs vertically. If flex-direction changes to column, these axes change direction.

justify-content — main-axis alignment

Use justify-content to distribute/align flex items along the main axis. Common values: flex-start, center, flex-end, space-between, space-around, and space-evenly.

```
#container {
  display: flex;
  gap: 20px;
  justify-content: center;
}
```

With the default row direction, justify-content: center centers items horizontally. flex-start places them at the start; flex-end places them at the end.

align-items — cross-axis alignment

Use align-items to align flex items on the cross axis. With a row layout, align-items: center aligns items vertically within the container's cross-axis space. A visible vertical-centering effect generally requires the container to have extra height.

```
#container {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 400px;
}
```

The instructor notes that a fixed container height helps demonstrate vertical centering. The exact layout also depends on the container's available space and its other CSS rules.

8. Flex property: distribute available space

The flex property controls how flex items grow/shrink and how available space is distributed. In the lecture's simple examples, flex: 1 on each box gives them equal shares of available flex space, while flex: 2 gives that item twice the flex share of an item with flex: 1, subject to the flex algorithm and any other sizing constraints.

```
.container {
  display: flex;
  gap: 20px;
}
.box { flex: 1; }       /* equal flex share */
.box-wide { flex: 2; } /* double share */
```

```
<div class="container">
  <div class="box">Box 1</div>
  <div class="box box-wide">Box 2</div>
  <div class="box">Box 3</div>
</div>
```

If the flex factors are 1, 2 and 1, the ratio is 1:2:1. In a simplified example without other constraints, the items receive roughly 25%, 50% and 25% of the distributable space. The gap is separate space between items.

Responsive layout: Flexbox can adapt item sizes as the container changes. It does not automatically make every layout mobile-friendly; use suitable sizing, wrapping, and sometimes media queries.

9. Suggested lab assignment from the lecture

Create a complete page with these sections, and choose a consistent color/font theme instead of random colors:

• Navbar: logo and brand name on the left; Home, About Us, Contact Us, Gallery or similar links on the right.

• Intro section: centered heading, a paragraph of about 3–4 lines, and one button.

• Two-column section: a smaller email-subscription form with a button on one side, and a wider descriptive paragraph on the other.

• Three equal-width sections: each has an image centered above a centered paragraph.

Use flexbox for the navbar and side-by-side sections. Use a parent container with display: flex; give the smaller/larger panels different flex values, and give the three final cards equal flex values. Consider flex-wrap or a media query for narrow screens.

10. JavaScript introduction

The lecture transitions from CSS to JavaScript. HTML defines structure/content; CSS styles it; JavaScript adds behavior and client-side logic so the page can respond to user actions.

Examples named in the lecture: calculations in the browser, taking decisions based on input, form validation, showing error messages, enabling/disabling a submit button, and adding animations or interactive updates.

```
<!-- Inline demonstration only -->
<button id="hello-button">Click me</button>
<p id="message"></p>

<script>
  document.getElementById("hello-button").addEventListener("click", function () {
    document.getElementById("message").textContent = "Hello!";
  });
</script>
```

This small example makes the page interactive: when the user clicks the button, JavaScript changes the paragraph text. The HTML creates the button and paragraph; JavaScript responds to the click.

11. JavaScript name and browser execution

The instructor explains that JavaScript was previously called LiveScript and that the JavaScript name was adopted during the language's early history. The key point for learning is that Java and JavaScript are separate languages despite the similar names and some familiar-looking syntax.

The lecture says a browser has a JavaScript engine that executes JavaScript. Examples of browsers mentioned include Chrome, Firefox, Brave, Opera and Edge. You can run basic JavaScript in the browser without separately installing a JavaScript compiler.

Extra clarification: Modern JavaScript engines often use both interpretation and just-in-time compilation internally, so "JavaScript has no compiler" is a beginner simplification. The practical point remains: for simple browser exercises, you can write JavaScript and run it in the browser without manually compiling it like a C/C++ program.

12. Dynamic typing, type safety and TypeScript

The transcript explains that JavaScript variables can hold values of different types at different times. For example, a variable can first hold a number and later hold a string. JavaScript has runtime types, but ordinary JavaScript variable declarations do not require a fixed type annotation.

```
let value = 5;
value = "five";  // valid JavaScript

console.log(typeof value); // "string"
```

The instructor warns that changing a variable's type unexpectedly can cause bugs and introduces TypeScript as a language built on JavaScript that adds static type checking. TypeScript code is typically checked/transformed into JavaScript before browsers execute it.

Important clarification: JavaScript absolutely has data types; it is dynamically typed. The more accurate distinction is that JavaScript normally does not enforce a declared variable type at compile time, while TypeScript can detect many type errors during development. JavaScript engines may compile code internally.

13. Quick revision sheet

■ Center a fixed-width block horizontally: margin-left/right auto, often shorthand margin: 80px auto 0.

■ text-align: center aligns inline content inside a container; it does not center the container itself.

■ position: static is normal flow; relative shifts while reserving space; absolute leaves normal flow; fixed usually attaches to the viewport.

■ top, left, right and bottom work as offsets depending on position mode.

■ Flexbox is applied to a parent; its direct children become flex items.

■ Default flex-direction is row: main axis horizontal, cross axis vertical.

■ justify-content aligns on the main axis; align-items aligns on the cross axis.

■ gap creates consistent space between flex items.

■ flex: 1 / flex: 2 demonstrates relative distribution of available flex space.

■ HTML = structure; CSS = appearance; JavaScript = behavior and client-side logic.

■ JavaScript is dynamically typed; TypeScript adds static type checking.

14. Practice questions

• What is the difference between margin: auto and text-align: center?

• Which position value keeps an element in normal flow but lets you visually offset it?

• What is the difference between relative and absolute positioning?

• Why is display: flex usually applied to the parent container?

• With flex-direction: row, which axis is the main axis?

• What is the difference between justify-content and align-items?

• What does gap: 20px do in a flex container?

• If three items have flex factors 1, 2 and 1, what is their simplified space ratio?

• Give two examples of client-side JavaScript logic.

• How are Java and JavaScript related? Are they the same language?

• What does dynamically typed mean in JavaScript?

• What does TypeScript add to JavaScript?

Self-check answers: 1) margin auto centers a block horizontally; text-align centers inline content. 2) relative. 3) relative retains its original flow space; absolute is removed from normal flow and positioned relative to a containing block. 4) the parent arranges its direct children. 5) horizontal. 6) main axis vs cross axis. 7) consistent spacing between items. 8) 1:2:1. 9) validation, calculations, click response, enabling/disabling submit. 10) separate languages. 11) a variable can hold values of different types at runtime. 12) static type checking/type annotations.